

Experiment ‘p_14_7_bubble_sort_seq’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_14_7_bubble_sort_seq

Natural language statement: null

Method signature: p_14_4_bubble_sort_seq(inputs: seq<int>) returns (sorted: seq<int>)

Ensures

- forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
- multiset(inputs) == multiset(sorted)

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_14_4_bubble_sort_seq(inputs: seq<int>) returns (sorted: seq<int>)`

The method should respect the following contract:

`ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j], ensures multiset
↪ (inputs) == multiset(sorted)`

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQA7UKzGFbm594FYtcj5ogjmtlbE

Model: gpt-5.5-2026-04-23

Created at: 1780671011

Finish reason: stop

Usage: 308 tokens in, and 6250 tokens out

```
{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    invariant multiset(sorted) == multiset(inputs[..n])
  {
    var x := inputs[n];
    var k := 0;
    while k < |sorted| && sorted[k] <= x
      invariant 0 <= k <= |sorted|
      invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
      invariant forall t :: 0 <= t < k ==> sorted[t] <= x
      invariant multiset(sorted) == multiset(inputs[..n])
    {
      k := k + 1;
    }
  }

  var old := sorted;
```

```

ghost var oldLen := |old|;
assert forall a, b :: 0 <= a < b < |old| ==> old[a] <= old[b];
assert forall t :: 0 <= t < k ==> old[t] <= x;
assert 0 <= k <= oldLen;
if k < oldLen {
  assert !(k < |old| && old[k] <= x);
  assert !(old[k] <= x);
  assert x < old[k];
}

sorted := old[..k] + [x] + old[k..];
assert |old[..k]| == k;
assert |old[..k] + [x]| == k + 1;
assert |sorted| == oldLen + 1;

assert forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j] by {
  forall i, j | 0 <= i < j < |sorted|
    ensures sorted[i] <= sorted[j]
  {
    if j < k {
      assert 0 <= i < j < |old|;
      assert old[i] <= old[j];
      assert sorted[i] == old[i];
      assert sorted[j] == old[j];
    } else if j == k {
      assert i < k;
      assert old[i] <= x;
      assert sorted[i] == old[i];
      assert sorted[j] == x;
    } else {
      assert k < j;
      if i < k {
        assert j - 1 < oldLen;
        assert k <= j - 1;
        assert i < j - 1;
        assert old[i] <= old[j - 1];
        assert sorted[i] == old[i];
        assert sorted[j] == old[j - 1];
      } else if i == k {
        assert j - 1 < oldLen;
        assert k <= j - 1;
        assert k < oldLen;
        assert x < old[k];
        if k < j - 1 {
          assert old[k] <= old[j - 1];
          assert x <= old[j - 1];
        } else {
          assert k == j - 1;
          assert x < old[j - 1];
        }
      }
      assert sorted[i] == x;
    }
  }
}

```

```

        assert sorted[j] == old[j - 1];
    } else {
        assert k < i;
        assert 0 <= i - 1 < j - 1 < oldLen;
        assert old[i - 1] <= old[j - 1];
        assert sorted[i] == old[i - 1];
        assert sorted[j] == old[j - 1];
    }
}
}
}

assert old == old[..k] + old[k..];
assert sorted == old[..k] + [x] + old[k..];
assert multiset(old) == multiset(old[..k]) + multiset(old[k..]);
assert multiset(sorted) == multiset(old[..k]) + multiset([x]) + multiset(old[k..]);
assert multiset(sorted) == multiset(old) + multiset([x]);
assert x == inputs[n];
assert inputs[..n + 1] == inputs[..n] + [x];
assert multiset(inputs[..n + 1]) == multiset(inputs[..n]) + multiset([x]);
assert multiset(sorted) == multiset(inputs[..n + 1]);

n := n + 1;
}
assert n == |inputs|;
assert inputs[..n] == inputs;
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```
problem_data/p_14_7_bubble_sort_seq/program.dfy(23,8): Error: invalid Ident
```

```

|
23 | var old := sorted;
| ^

```

```
problem_data/p_14_7_bubble_sort_seq/program.dfy(24,28): Error: openparen expected
```

```

|
24 | ghost var oldLen := |old|;
| ^

```

```
problem_data/p_14_7_bubble_sort_seq/program.dfy(25,43): Error: openparen expected
```

```

|
25 | assert forall a, b :: 0 <= a < b < |old| ==> old[a] <= old[b];
| ^

```

```
problem_data/p_14_7_bubble_sort_seq/program.dfy(25,52): Error: openparen expected
```

```

|
25 | assert forall a, b :: 0 <= a < b < |old| ==> old[a] <= old[b];
| ^

```

```

problem_data/p_14_7_bubble_sort_seq/program.dfy(25,62): Error: openparen expected
|
25 | assert forall a, b :: 0 <= a < b < |old| ==> old[a] <= old[b];
| ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(25,65): Error: closeparen expected
|
25 | assert forall a, b :: 0 <= a < b < |old| ==> old[a] <= old[b];
| ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(26,41): Error: openparen expected
|
26 | assert forall t :: 0 <= t < k ==> old[t] <= x;
| ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(26,49): Error: closeparen expected
|
26 | assert forall t :: 0 <= t < k ==> old[t] <= x;
| ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(29,23): Error: openparen expected
|
29 | assert !(k < |old| && old[k] <= x);
| ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(29,31): Error: openparen expected
|
29 | assert !(k < |old| && old[k] <= x);
| ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(29,40): Error: verticalbar expected
|
29 | assert !(k < |old| && old[k] <= x);
| ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(30,18): Error: openparen expected
|
30 | assert !(old[k] <= x);
| ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(30,27): Error: closeparen expected
|
30 | assert !(old[k] <= x);
| ^

problem_data/p_14_7_bubble_sort_seq/program.dfy(31,20): Error: openparen expected
|
31 | assert x < old[k];
| ^

```

```
problem_data/p_14_7_bubble_sort_seq/program.dfy(31,23): Error: closeparen expected
|
31 | assert x < old[k];
| ^
```

```
problem_data/p_14_7_bubble_sort_seq/program.dfy(34,17): Error: openparen expected
|
34 | sorted := old[..k] + [x] + old[k..];
| ^
```

16 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
→ Dafny body, including the curly braces that surround it. Do not show the signature
→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQCGPflwNteU7uL2ScyYmnfljKvr

Model: gpt-5.5-2026-04-23

Created at: 1780671144

Finish reason: stop

Usage: 2979 tokens in, and 2056 tokens out

```
{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    invariant multiset(sorted) == multiset(inputs[..n])
  {
    var x := inputs[n];
    var k := 0;
    while k < |sorted| && sorted[k] <= x
      invariant 0 <= k <= |sorted|
      invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
      invariant forall t :: 0 <= t < k ==> sorted[t] <= x
      invariant multiset(sorted) == multiset(inputs[..n])
    {
      k := k + 1;
    }

    var prev := sorted;
    ghost var prevLen := |prev|;
    assert forall a, b :: 0 <= a < b < |prev| ==> prev[a] <= prev[b];
    assert forall t :: 0 <= t < k ==> prev[t] <= x;
    assert 0 <= k <= prevLen;
    if k < prevLen {
      assert !(k < |prev| && prev[k] <= x);
    }
  }
}
```

```

    assert !(prev[k] <= x);
    assert x < prev[k];
}

sorted := prev[..k] + [x] + prev[k..];
assert |prev[..k]| == k;
assert |prev[..k] + [x]| == k + 1;
assert |sorted| == prevLen + 1;

assert forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j] by {
  forall i, j | 0 <= i < j < |sorted|
    ensures sorted[i] <= sorted[j]
{
  if j < k {
    assert 0 <= i < j < |prev|;
    assert prev[i] <= prev[j];
    assert sorted[i] == prev[i];
    assert sorted[j] == prev[j];
  } else if j == k {
    assert i < k;
    assert prev[i] <= x;
    assert sorted[i] == prev[i];
    assert sorted[j] == x;
  } else {
    assert k < j;
    if i < k {
      assert j - 1 < prevLen;
      assert k <= j - 1;
      assert i < j - 1;
      assert prev[i] <= prev[j - 1];
      assert sorted[i] == prev[i];
      assert sorted[j] == prev[j - 1];
    } else if i == k {
      assert j - 1 < prevLen;
      assert k <= j - 1;
      assert k < prevLen;
      assert x < prev[k];
      if k < j - 1 {
        assert prev[k] <= prev[j - 1];
        assert x <= prev[j - 1];
      } else {
        assert k == j - 1;
        assert x < prev[j - 1];
      }
    }
    assert sorted[i] == x;
    assert sorted[j] == prev[j - 1];
  } else {
    assert k < i;
    assert 0 <= i - 1 < j - 1 < prevLen;
    assert prev[i - 1] <= prev[j - 1];
    assert sorted[i] == prev[i - 1];
  }
}

```

```

        assert sorted[j] == prev[j - 1];
    }
}
}

assert prev == prev[..k] + prev[k..];
assert sorted == prev[..k] + [x] + prev[k..];
assert multiset(prev) == multiset(prev[..k]) + multiset(prev[k..]);
assert multiset(sorted) == multiset(prev[..k]) + multiset([x]) + multiset(prev[k..]);
assert multiset(sorted) == multiset(prev) + multiset([x]);
assert x == inputs[n];
assert inputs[..n + 1] == inputs[..n] + [x];
assert multiset(inputs[..n + 1]) == multiset(inputs[..n]) + multiset([x]);
assert multiset(sorted) == multiset(inputs[..n + 1]);

n := n + 1;
}
assert n == |inputs|;
assert inputs[..n] == inputs;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_14_7_bubble_sort_seq/program.dfy(1,7): Error: Verification of '
  ↪ p_14_4_bubble_sort_seq' timed out after 31 seconds. (the limit can be increased
  ↪ using --verification-time-limit)
|
1 | method p_14_4_bubble_sort_seq(inputs: seq<int>) returns (sorted: seq<int>)
|   ~~~~~

```

Dafny program verifier finished with 1 verified, 0 errors, 1 time out

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQDkAbwxDj77iltVxwu34SAX5eJ5

Model: gpt-5.5-2026-04-23

Created at: 1780671236

Finish reason: stop

Usage: 3545 tokens in, and 4845 tokens out

```

{
  sorted := [];

```



```

var rest := inputs;

while |rest| > 0
  invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  invariant forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] <= rest[j]
  invariant multiset(inputs) == multiset(sorted) + multiset(rest)
  decreases |rest|
{
  assert 0 < |rest|;
  var minIdx := 0;
  var scan := 1;
  assert forall t :: 0 <= t < 1 ==> rest[0] <= rest[t];

  while scan < |rest|
    invariant 0 <= minIdx < |rest|
    invariant 1 <= scan <= |rest|
    invariant minIdx < scan
    invariant forall t :: 0 <= t < scan ==> rest[minIdx] <= rest[t]
    decreases |rest| - scan
  {
    var prevMin := minIdx;
    if rest[scan] < rest[prevMin] {
      minIdx := scan;
      assert forall t :: 0 <= t < scan ==> rest[minIdx] <= rest[t] by {
        forall t | 0 <= t < scan
          ensures rest[minIdx] <= rest[t]
        {
          assert rest[prevMin] <= rest[t];
          assert rest[minIdx] == rest[scan];
          assert rest[scan] < rest[prevMin];
        }
      }
    } else {
      assert minIdx == prevMin;
      assert rest[minIdx] <= rest[scan];
    }

    assert forall t :: 0 <= t < scan + 1 ==> rest[minIdx] <= rest[t] by {
      forall t | 0 <= t < scan + 1
        ensures rest[minIdx] <= rest[t]
      {
        if t < scan {
          assert rest[minIdx] <= rest[t];
        } else {
          assert t == scan;
          assert rest[minIdx] <= rest[scan];
        }
      }
    }
    scan := scan + 1;
  }
}

```

```

assert scan == |rest|;
var m := rest[minIdx];
assert forall t :: 0 <= t < |rest| ==> m <= rest[t];

var priorSorted := sorted;
var priorRest := rest;
ghost var priorSortedLen := |priorSorted|;
ghost var priorRestLen := |priorRest|;

sorted := priorSorted + [m];
rest := priorRest[..minIdx] + priorRest[minIdx + 1..];

assert |sorted| == priorSortedLen + 1;
assert |rest| == priorRestLen - 1;

assert forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j] by {
  forall i, j | 0 <= i < j < |sorted|
    ensures sorted[i] <= sorted[j]
  {
    if j < priorSortedLen {
      assert sorted[i] == priorSorted[i];
      assert sorted[j] == priorSorted[j];
      assert priorSorted[i] <= priorSorted[j];
    } else {
      assert j == priorSortedLen;
      assert i < priorSortedLen;
      assert sorted[i] == priorSorted[i];
      assert sorted[j] == m;
      assert priorSorted[i] <= priorRest[minIdx];
    }
  }
}

assert forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] <= rest[j] by
  ↪ {
  forall i, j | 0 <= i < |sorted| && 0 <= j < |rest|
    ensures sorted[i] <= rest[j]
  {
    if j < minIdx {
      assert rest[j] == priorRest[j];
      if i < priorSortedLen {
        assert sorted[i] == priorSorted[i];
        assert priorSorted[i] <= priorRest[j];
      } else {
        assert i == priorSortedLen;
        assert sorted[i] == m;
        assert m <= priorRest[j];
      }
    } else {
      assert j + 1 < priorRestLen;

```

```

    assert rest[j] == priorRest[j + 1];
    if i < priorSortedLen {
      assert sorted[i] == priorSorted[i];
      assert priorSorted[i] <= priorRest[j + 1];
    } else {
      assert i == priorSortedLen;
      assert sorted[i] == m;
      assert m <= priorRest[j + 1];
    }
  }
}

assert priorRest == priorRest[..minIdx] + [m] + priorRest[minIdx + 1..];
assert multiset(priorRest) == multiset(priorRest[..minIdx]) + multiset([m]) + multiset
  ↪ (priorRest[minIdx + 1..]);
assert multiset(rest) == multiset(priorRest[..minIdx]) + multiset(priorRest[minIdx +
  ↪ 1..]);
assert multiset(priorRest) == multiset(rest) + multiset([m]);
assert multiset(sorted) == multiset(priorSorted) + multiset([m]);
assert multiset(inputs) == multiset(priorSorted) + multiset(priorRest);
assert multiset(sorted) + multiset(rest) == multiset(inputs);
assert multiset(inputs) == multiset(sorted) + multiset(rest);
}

assert |rest| == 0;
assert rest == [];
assert multiset(inputs) == multiset(sorted) + multiset(rest);
assert multiset(inputs) == multiset(sorted);
}

```

Final Program

```

method p_14_4_bubble_sort_seq(inputs: seq<int>) returns (sorted: seq<int>)
  ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  ensures multiset(inputs) == multiset(sorted)
{
  sorted := [];
  var rest := inputs;

  while |rest| > 0
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    invariant forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] <= rest[j]
    invariant multiset(inputs) == multiset(sorted) + multiset(rest)
    decreases |rest|
  {
    assert 0 < |rest|;
    var minIdx := 0;
    var scan := 1;
    assert forall t :: 0 <= t < 1 ==> rest[0] <= rest[t];

```

```

while scan < |rest|
  invariant 0 <= minIdx < |rest|
  invariant 1 <= scan <= |rest|
  invariant minIdx < scan
  invariant forall t :: 0 <= t < scan ==> rest[minIdx] <= rest[t]
  decreases |rest| - scan
{
  var prevMin := minIdx;
  if rest[scan] < rest[prevMin] {
    minIdx := scan;
    assert forall t :: 0 <= t < scan ==> rest[minIdx] <= rest[t] by {
      forall t | 0 <= t < scan
        ensures rest[minIdx] <= rest[t]
      {
        assert rest[prevMin] <= rest[t];
        assert rest[minIdx] == rest[scan];
        assert rest[scan] < rest[prevMin];
      }
    }
  } else {
    assert minIdx == prevMin;
    assert rest[minIdx] <= rest[scan];
  }

  assert forall t :: 0 <= t < scan + 1 ==> rest[minIdx] <= rest[t] by {
    forall t | 0 <= t < scan + 1
      ensures rest[minIdx] <= rest[t]
    {
      if t < scan {
        assert rest[minIdx] <= rest[t];
      } else {
        assert t == scan;
        assert rest[minIdx] <= rest[scan];
      }
    }
  }
  scan := scan + 1;
}

assert scan == |rest|;
var m := rest[minIdx];
assert forall t :: 0 <= t < |rest| ==> m <= rest[t];

var priorSorted := sorted;
var priorRest := rest;
ghost var priorSortedLen := |priorSorted|;
ghost var priorRestLen := |priorRest|;

sorted := priorSorted + [m];
rest := priorRest[..minIdx] + priorRest[minIdx + 1..];

```

```

assert |sorted| == priorSortedLen + 1;
assert |rest| == priorRestLen - 1;

assert forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j] by {
  forall i, j | 0 <= i < j < |sorted|
    ensures sorted[i] <= sorted[j]
  {
    if j < priorSortedLen {
      assert sorted[i] == priorSorted[i];
      assert sorted[j] == priorSorted[j];
      assert priorSorted[i] <= priorSorted[j];
    } else {
      assert j == priorSortedLen;
      assert i < priorSortedLen;
      assert sorted[i] == priorSorted[i];
      assert sorted[j] == m;
      assert priorSorted[i] <= priorRest[minIdx];
    }
  }
}

assert forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] <= rest[j] by
  ↪ {
  forall i, j | 0 <= i < |sorted| && 0 <= j < |rest|
    ensures sorted[i] <= rest[j]
  {
    if j < minIdx {
      assert rest[j] == priorRest[j];
      if i < priorSortedLen {
        assert sorted[i] == priorSorted[i];
        assert priorSorted[i] <= priorRest[j];
      } else {
        assert i == priorSortedLen;
        assert sorted[i] == m;
        assert m <= priorRest[j];
      }
    } else {
      assert j + 1 < priorRestLen;
      assert rest[j] == priorRest[j + 1];
      if i < priorSortedLen {
        assert sorted[i] == priorSorted[i];
        assert priorSorted[i] <= priorRest[j + 1];
      } else {
        assert i == priorSortedLen;
        assert sorted[i] == m;
        assert m <= priorRest[j + 1];
      }
    }
  }
}

```

```

    assert priorRest == priorRest[..minIdx] + [m] + priorRest[minIdx + 1..];
    assert multiset(priorRest) == multiset(priorRest[..minIdx]) + multiset([m]) + multiset
      ↪ (priorRest[minIdx + 1..]);
    assert multiset(rest) == multiset(priorRest[..minIdx]) + multiset(priorRest[minIdx +
      ↪ 1..]);
    assert multiset(priorRest) == multiset(rest) + multiset([m]);
    assert multiset(sorted) == multiset(priorSorted) + multiset([m]);
    assert multiset(inputs) == multiset(priorSorted) + multiset(priorRest);
    assert multiset(sorted) + multiset(rest) == multiset(inputs);
    assert multiset(inputs) == multiset(sorted) + multiset(rest);
  }

  assert |rest| == 0;
  assert rest == [];
  assert multiset(inputs) == multiset(sorted) + multiset(rest);
  assert multiset(inputs) == multiset(sorted);
}

```

Total Token Usage

Input tokens: 6832

Output tokens: 13151

Reasoning tokens: 9535

Sum of 'total tokens': 19983

Experiment Timings

Overall Experiment started at 1780671009638, ended at 1780671363236, lasting 353598ms (353.60 seconds)

Iteration #1 started at 1780671009843, ended at 1780671142181, lasting 132338ms (132.34 seconds)

Iteration #2 started at 1780671142182, ended at 1780671235676, lasting 93494ms (93.49 seconds)

Iteration #3 started at 1780671235689, ended at 1780671363236, lasting 127547ms (127.55 seconds)